

main.py:

Order of implementation of the modules:

camera.py -> detector.py -> ultrasonic.py -> kalman_filter.py -> caption.py -> speaker.py -> buzzer.py

Libraries explanation:

os → Interact with the operating system.

- Create/delete folders
- Work with file paths
- Run system commands

sys → Access Python interpreter and command-line arguments.

- Read arguments (sys.argv)
- Exit programs (sys.exit())
- Access Python version (sys.version)

time → Work with time and delays.

- Pause execution (time.sleep(5))
- Get current timestamp (time.time())
- Measure execution time

threading

cv2

Qt (cute) is a framework for creating GUIs: it provides windows, buttons, menus, dialog boxes, image/video displays. OpenCV's cv2.imshow() makes use of this. It is handling the window display of our Rpi.

Wayland: modern display system for linux

Qt -> creates windows/buttons, etc. -> wayland displays the server protocol (puts everything on screen)

In our case, wayland we are using X11

Importing order:

```
import config
from camera import Camera
from ultrasonic import UltrasonicSensor
from detector import ObjectDetector
from kalman_filter import KalmanFilter1D
from caption import CaptionGenerator
from speaker import Speaker
from buzzer import Buzzer
```

Config.py:

Contains pin assignments, camera settings (camera index = 0, width: 320, length: 240, fps = 15), object detection (path to prototxt and caffemodel files)

Detection confidence threshold: taken 0.55

List of all classes to be detected -> filtered from 20 to 13

Kalman filter parameters: since kalman filter is used to estimate true positions of something when measurements are noisy, we have some parameters defined to help us with the process.

Process noise/variance = 0.05 (small value = small movement expected)

Measurement noise = 5.0 (noise in sensor, higher value = lesser trust on sensor readings)

Initial estimate = 100 (start guessing for distance before any reading)

Initial error = 50 (uncertainty in initial goes)

Outlier threshold = 80

Ultrasonic Sensor Settings: max distance = 4m, timeout = 0.04 (wait at most 0.04 seconds for echo to return, preventing the code from hanging forever if no echo is received), read interval = 0.05 (20 readings per second)

Buzzer alert distance tiers (cm): off, slow, medium, fast

TTS Rate = 160 (words per minute)

TTS Volume = 100%

Caption settings:

Caption cooldown = 3 seconds

Caption close threshold = 50cm (triggers "Warning!" Prefix)

Caption very close = 20cm (triggers urgent alert)

Show_display = True (Show OpenCV window)

Main_loop_delay = 0.1 (seconds between main loop iterations)

Camera.py:

A wrapper "Camera" has been defined that can make it easy for us to implement functions like "start", "get_frame" and "stop".

Attributes defined: index (can be taken from config.CAMERA_INDEX), width (can be taken from config.CAMERA_WIDTH), height (can be taken from config.CAMERA_HEIGHT), fps (can be taken from config.CAMERA_FPS), cap (Camera capture object/variable)

Methods defined: start, get_frame, stop, isopened

NOTE: Can remove the Qt/wayland part in config

Ultrasonic.py:

Created an “UltrasonicSensor” object with attributes: trig_pin, echo_pin, max_distance, timeout, read_interval, _distance, _lock, _running, _thread

Methods defined: _setup_gpio, _read_once, _background_loop, start, stop, get_distance, cleanup

Detector.py

Here we are detecting the object using the MobileNet-SSD caffe model via OpenCV DNN

Running inferences on camera frames, returning detected objects with class names, confidence scores and bounding boxes

20 PASCAL VOC Classes -> 13 filtered ones

Created an object called “ObjectDetector” with attributes: confidence_threshold, class_labels, net

Methods: load_model,

Input Image



MobileNet (CNN feature extractor)



SSD (object detection head)



Bounding boxes + class predictions

- **MobileNet** = lightweight CNN that extracts image features.
- **SSD (Single Shot Detector)** = uses those features to detect and classify objects in one pass.

Kalman_filter.py:

What's kalman filter? ->

Speed evolves like the following: $x(t+1) = A * x(t) + u$

In the above, u is a gaussian noise that may be present in the surroundings

Sometimes there may be multiple estimates of an object's position wrt initial position. We need to be certain about the noise to be able to select the best estimate. Hence,

Kalman Filters is an algorithm to estimate the state of the system (like speed and position) using past and noise observations and current measurements.

$x(t+1) = A * x(t) + u$ -> underlying dynamical system
 $y(t) = H * x(t) + v$ -> measurement that we get at time t

H is a row vector = [1 0] (Because we are measuring only the position)

Kalman filter is a two step process:-

1. Prediction Step -> you take prior estimates about state of the system, we run it through idealised dynamical system (first equation) and we get a new prediction for t + 1.

2. Update Step -> We take new measurements from above step which are noisy and we combine both in a single estimate which is our final estimate of the system.

Now we calculate the surprise term (difference between H * predicted term and measured term). Bigger the term, more surprised you are. Based on the surprise term, we get to know how much we need to correct our prediction. Our trust on the prediction can be checked by the covariance matrix $\Sigma(t+1)$.

We need to compute kalman ratio (K) = $\Sigma_{xx}/(\Sigma_{xx} + Q_{xx})$ belongs to [0,1]

In the above Σ_{xx} is the uncertainty term about position's prediction and Q_{xx} is uncertainty is about measurement. If K = 0, it means Σ_{xx} is zero and we are 100% sure our prediction is correct.

If K = 1, it means Q_{xx} is zero and we are 100% sure that our measurement is correct.

So we take our prediction $x'(t+1) = x(t+1) + K * \text{surprise term}$ (bigger the K, bigger the correction)

Created an object called "KalmanFilter1D" with multiple attributes:

Attribute	Meaning
<code>`self.x`</code>	estimated distance + velocity
<code>`self.P`</code>	uncertainty in estimate
<code>`self.R`</code>	sensor noise (ultrasonic error)
<code>`self.q`</code>	motion randomness
<code>`self.H`</code>	tells what we measure (distance only)
<code>`self.outlier_threshold`</code>	ignore crazy jumps
<code>`self._last_time`</code>	used to compute dt
<code>`self._initialized`</code>	first-time setup flag

Methods defined: predict, update, get_distance, get_velocity, reset

Caption.py:

Created a class "CaptionGenerator" with attributes: cooldown, close_threshold, very_close, _last_caption and _last_time

Methods defined are: _format_distance, _build_caption, generate, _get_tier, generate_proximity_warning

Speaker.py:

"Speaker" object with attributes: _queue, _running, _thread, _tts_cmd

Methods: start, _worker, speak, stop, is_speaking

Buzzer.py:

"Buzzer" class with attributes: pin, _distance, _lock, _running, _thread, _pwm

Method: _tone, _get_params, _background_loop, start, stop, update, cleanup